

Desarrollo entorno Cliente

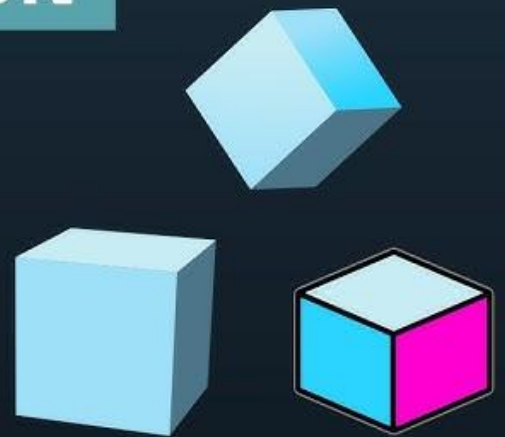
Programación Orientada a Objetos JavaScript

PROGRAMACIÓN

ORIENTADA

A OBJETOS

JS



PabloSG

Índice

1.Título del tema	2
2.Explicación detallada del contenido	2
2.1 Conceptos incluidos	2
2.2 Delimitación del tema	2
2.3 Nivel previsto de dificultad	3
3.Relación con la asignatura	3
4.Estructura prevista de la píldora	3
4.1 Introducción	3
4.2 Conceptos básicos	3
4.3 Base teórica	3
4.4 Ejemplos prácticos	3
4.5 Aplicación práctica/demostración	4
4.6 Turno de preguntas	4
5. Progresión del contenido	4

1.Título del tema

Introducción de la Programación Orientada a Objetos en JavaScript

2.Explicación detallada del contenido

La píldora formativa abordará los fundamentos de la Programación Orientada a Objetos (POO) en JavaScript, mostrando cómo organizar el código en torno a objetos que combinan datos (propiedades) y acciones (métodos).

El objetivo es que el alumno comprenda cómo aplicar POO en JavaScript para crear programas más claros, reutilizables y fáciles de mantener, especialmente en proyectos de mayor tamaño.

2.1 Conceptos incluidos

- Definición de **POO** y su importancia en JavaScript.
- Palabras clave esenciales: **class**, **constructor**, **new**, **extends**, **this**.
- Creación de **clases y objetos**.
- **Métodos dentro de las clases**.
- **Getters y Setters** para controlar el acceso a propiedades.
- **Sobrescritura de métodos heredados** (toString, valueOf).
- **Interfaces** (simulación mediante documentación o con TypeScript).
- Subclases y herencia.
- **Statics y Abstracts**: clases, interfaces, métodos y atributos.
- **Campos privados y públicos** en clases.
- **Mixins y composición** como alternativa a la herencia múltiple.
- **Variables asociadas a objetos**.
- **Iteradores y Generadores** dentro de clases.
- **Ejemplos prácticos de uso**.
- Problemas comunes al no aplicar POO.
- Analogías con la vida real para facilitar la comprensión.
- **Encapsulación y visibilidad**: campos privados (#) y diferencias con Java.
- Uso del operador **instanceof** y de **Object.create()** para herencia prototípica.
- Métodos especiales de objetos: **hasOwnProperty**, **Object.assign**, **structuredClone**.
- **Métodos asíncronos** (async) dentro de clases.

2.2 Delimitación del tema

- **Incluye:** conceptos básicos y extendidos de POO en JavaScript, ejemplos de clases, objetos, herencia, métodos, encapsulación y uso de iteradores/generadores.
- **Excluye:** patrones avanzados de diseño, polimorfismo complejo, frameworks externos, implementación completa de interfaces en TypeScript.

2.3 Nivel previsto de dificultad

- **Nivel básico-intermedio:** pensado para estudiantes que ya conocen las bases de JavaScript (variables, funciones, estructuras de control) y quieren dar el salto a organizar su código con POO, con una introducción a conceptos más avanzados de manera progresiva.

3.Relación con la asignatura

El contenido de la píldora se centra en la creación y uso de clases, objetos y herencia en JavaScript, reforzando la base teórica y práctica del módulo DWEK y preparando al alumno para abordar proyectos más complejos con un código mejor estructurado.

4.Estructura prevista de la píldora

4.1 Introducción

La **POO** usa objetos para organizar el código.

4.2 Conceptos básicos

Clase, constructor, métodos, subclases, variables, encapsulación, iteradores, métodos especiales.

4.3 Base teórica

Explicación detallada con ejemplos de código.

4.4 Ejemplos prácticos

- Creación de una clase básica.
- Creación de objetos y subclases.
- Ejecución de métodos.
- Uso de getters/setters y métodos asíncronos.

4.5 Aplicación práctica/demostración

Ejemplo de cómo organizar un pequeño programa con POO, integrando herencia, encapsulación y composición.

4.6 Turno de preguntas

Resolución de dudas y aclaraciones.

5. Progresión del contenido

- **Inicio:** Conceptos básicos de POO y su relación con la vida real.
- **Desarrollo:** Explicación de clases, constructores, métodos, herencia y encapsulación con ejemplos sencillos.
- **Avance:** Aplicación práctica con código completo, incluyendo getters/setters, iteradores y métodos asíncronos.
- **Cierre:** Resumen de los beneficios de la POO y cómo ayuda a mantener el código ordenado, reutilizable y escalable.